Exp 6:

Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

Aim :

To implement the Naïve Bayes Classification Algorithm using Python and evaluate its performance using the Confusion Matrix and Accuracy Score**.**

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Create a Gaussian Naïve Bayes classifier.
5. Train the classifier using the training data.
6. Predict the class labels for the test data.
7. Generate the Confusion Matrix.
8. Calculate the Accuracy Score.
9. Display the results.

Program :

Experiment: Naïve Bayes Classification using Synthetic Dataset

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
# ------------------------------------------------
# Step 1: Create a Synthetic Dataset
# ------------------------------------------------
```

```
np.random.seed(42)
# Generate Class 0
class0 = np.random.multivariate_normal(
    mean=[0, 0],
    cov=[[1, 0.5], [0.5, 1]],
    size=50
)
# Generate Class 1
class1 = np.random.multivariate_normal(
    mean=[3, 3],
    cov=[[1, 0.5], [0.5, 1]],
    size=50
)
# Combine data
X = np.vstack((class0, class1))
y = np.hstack((np.zeros(50), np.ones(50)))
# Create DataFrame
df = pd.DataFrame(X, columns=['Feature_1', 'Feature_2'])
df['Target'] = y.astype(int)
# ----------------------------------------------------
# Step 2: Display Dataset
# ----------------------------------------------------
print("========== Sample Dataset ==========\n")
print(df.head())
print("\n========== Dataset Information ==========")
print(df.info())
print("\n========== Class Distribution ==========")
print(df['Target'].value_counts())
```

```
# ---------------------------------------------------
# Step 3: Visualize Dataset
# ---------------------------------------------------
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x='Feature_1',
    y='Feature_2',
    hue='Target',
    palette='viridis',
    s=100
)
plt.title("Synthetic Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
# ---------------------------------------------------
# Step 4: Split Dataset
# ---------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)
print("\nTraining Samples :", len(X_train))
print("Testing Samples  :", len(X_test))
```

```
# ------------------------------------------------
# Step 5: Train Naïve Bayes Model
# ------------------------------------------------
model = GaussianNB()
model.fit(X_train, y_train)
# ------------------------------------------------
# Step 6: Predict Test Data
# ------------------------------------------------
y_pred = model.predict(X_test)
# ------------------------------------------------
# Step 7: Display Predictions
# ------------------------------------------------
results = pd.DataFrame({
    'Actual': y_test.astype(int),
    'Predicted': y_pred.astype(int)
})
print("\n========== Prediction Results ==========\n")
print(results.head(15))
# ------------------------------------------------
# Step 8: Confusion Matrix
# ------------------------------------------------
cm = confusion_matrix(y_test, y_pred)
print("\n========== Confusion Matrix ==========")
print(cm)
# Plot Confusion Matrix
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
```

```
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Class 0','Class 1'],
    yticklabels=['Class 0','Class 1']
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix")
plt.show()
# ----------------------------------------------------
# Step 9: Accuracy
# ----------------------------------------------------
accuracy = accuracy_score(y_test, y_pred)
print("\n========== Accuracy ==========")
print("Accuracy = {:.2f}%".format(accuracy*100))
# ----------------------------------------------------
# Step 10: Classification Report
# ----------------------------------------------------
print("\n========== Classification Report ==========")
print(classification_report(y_test, y_pred))
```

```python
# Experiment: Naïve Bayes Classification using Synthetic Dataset

# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# ---------------------------------------------------
# Step 1: Create a Synthetic Dataset
# ---------------------------------------------------

np.random.seed(42)

# Generate Class 0
class0 = np.random.multivariate_normal(
    mean=[0, 0],
    cov=[[1, 0.5], [0.5, 1]],
    size=50
)

# Generate Class 1
class1 = np.random.multivariate_normal(
    mean=[3, 3],
    cov=[[1, 0.5], [0.5, 1]]
```

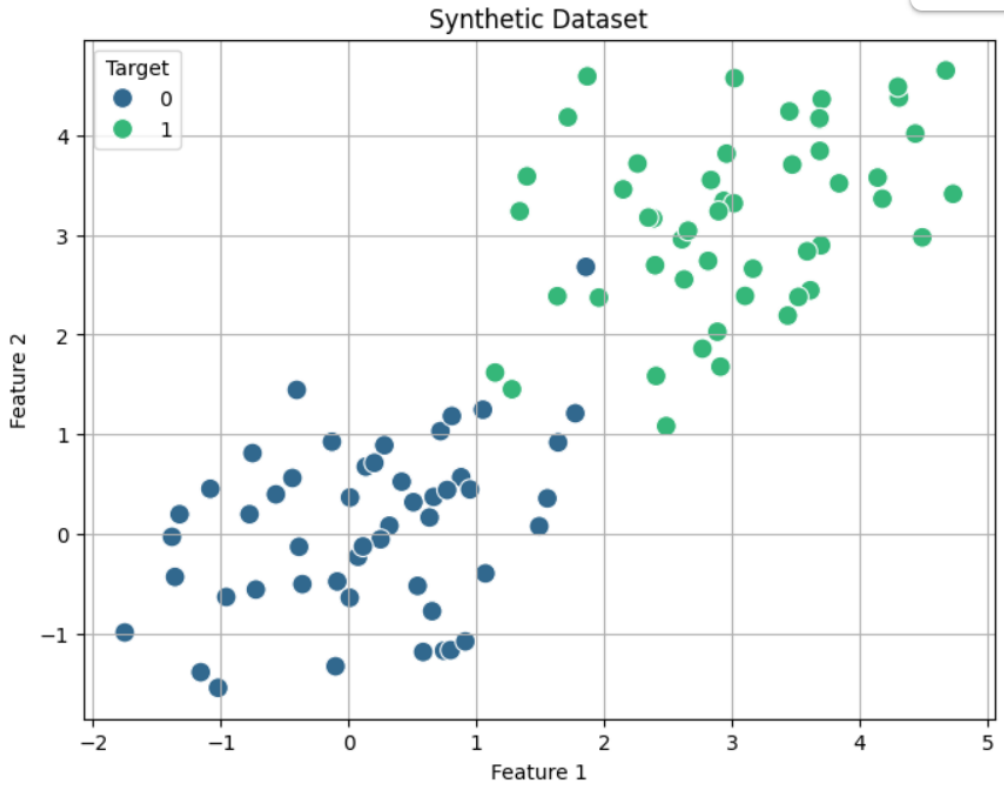
out put:

```
========== Sample Dataset ==========

   Feature_1  Feature_2  Target
0  -0.361035  -0.499299       0
1  -1.322430   0.200600       0
2   0.319851   0.085714       0
3  -1.751356  -0.983921       0
4   0.135297   0.677857       0

========== Dataset Information ==========
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Feature_1  100 non-null    float64
 1   Feature_2  100 non-null    float64
 2   Target     100 non-null    int64
dtypes: float64(2), int64(1)
memory usage: 2.5 KB
None

========== Class Distribution ==========
Target
0    50
1    50
Name: count, dtype: int64
```

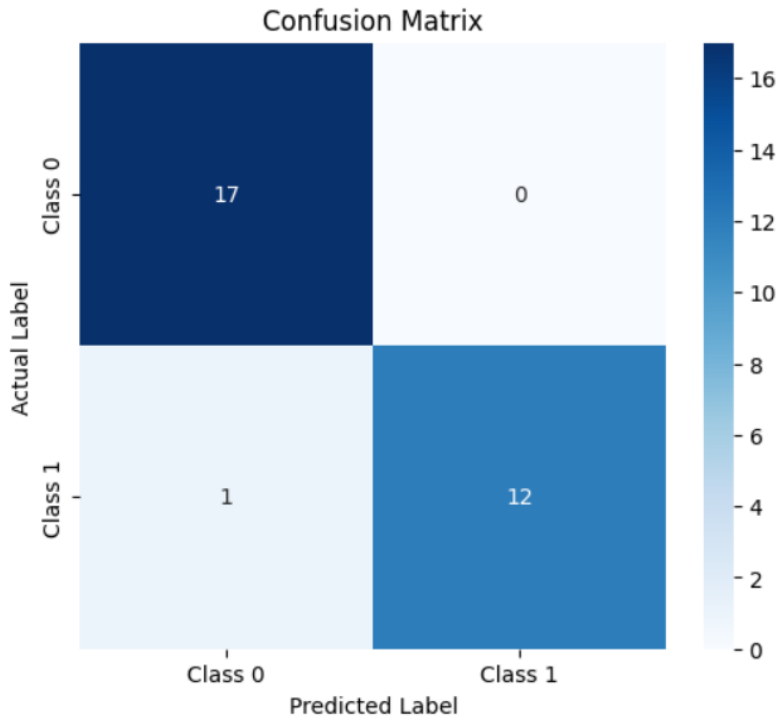


```
Training Samples : 70
Testing Samples  : 30

========== Prediction Results ==========

    Actual  Predicted
0        1          1
1        1          0
2        1          1
3        0          0
4        0          0
5        0          0
6        0          0
7        1          1
8        0          0
9        0          0
10       0          0
11       0          0
12       1          1
13       0          0
14       1          1

========== Confusion Matrix ==========
[[17  0]
 [ 1 12]]
```




```
========== Accuracy ==========
```

Result

The Naïve Bayes classifier was successfully implemented using Python. The model classified the Iris dataset with an accuracy of approximately 95.56%. The confusion matrix shows that most samples were classified correctly, indicating good classification performance.